primepayments

MERCHANT API DOCUMENT

QUERY TRANSACTION

Table of Contents

Table of Contents **1**
Document Release Note **2**
Prerequisite **3**
Merchant Account 3
Query Transaction **4**
Request Parameters 4
Example 5
Response Parameters 6
Example 10
Appendix A - Message Encryption **12**
Example 12
PHP 7 12
JAVA 8 12
Appendix B - Message Decryption **15**
Example 15
PHP 7 15
JAVA 8 16

Document Release Note

Version	Date	Description	Author
1.0.0	4 Feb 2021	First version	K5

Prerequisite

Merchant Account

When merchant account is ready, we will provide merchant the followings

1) Merchant Back Office URL
2) API Endpoint URL
3) Merchant ID (16 characters, alphanumeric unique ID to identify merchant)
4) Merchant Key (16 characters, secret key for encryption)
5) Merchant Hash IV (16 characters, the hash IV (Initiation Vector) used for encryption)

Query Transaction

Test Environment
API Endpoint : http://api-test.primepayco.com/api/merchant/query
HTTP Method : POST
HTTP Header : Content-Type: application/json
Request Data Format : JSON
Response Data Format : JSON

Request Parameters

Field ID	Field Name	Data Type	Description	Example Value
merchantId		Char (16)	This will be provided	
message			Refer Message details in the table below. The message in json format is encrypted json data, refer the Error! Not a valid bookmark self-reference. section below for the data fields	

Message (in JSON Format)

Field ID	Field Name	Data Type	Description	Example Value
Mandatory				
merchantId		Char (16)	This will be provided	
referenceId		Char (45)	The unique deposit/withdrawal request transaction ID from us for merchant reference	
transactionId		Char (45)	**When referenceId is absent, this is mandatory** A unique reference from merchant to identify the fund In/deposit request transaction.	

Example

Body Data (JSON Format)


```
{
	"merchantId":"94b2522ba609909d",
	"message":"qm0PEHj02XzwMMWGM4j2S/Y1GWcV4BowAZFrUNDI8kH51QwkK/dF+FeGxnsgw1PXQWNwnt16pfFesIlRfh/6/GrMO4FFGAiW2fi6P5
Zn1zRYr/+8QJm/sYvKcuANipP8"
}
```


Message before encrypted


```
{
	"merchantId": "94b2522ba609909d",
	"referenceId": "855112b2-a88b-49a5-8b82-a6b09d7db487"
}
```

Response Parameters

Field ID	Field Name	Data Type	Description	Example Value
status		Char (10)		success or fail
When status = success				
result		Text	Refer Result details in the table below. The result in json format is encrypted json data, refer the **Appendix B - Message** Decryption section below to descript the string in order to obtain the json format data.	
When status = fail				
errors		Text	When 1) Incorrect Parameters 2) There is no bank account available.	When status = fail

Result (in JSON Format)

Field ID	Field Name	Data Type	Description	Example Value
Mandatory				
referenceId		Char (45)	The unique fund out/withdrawal request transaction ID from us for merchant reference	
transactionId		Char (45)	A unique reference from merchant to identify the fund In/deposit request transaction.	
status		Integer	The request status	Possible values **1**: Completed **2**: Rejected **4**: Refunded

Field ID	Field Name	Data Type	Description	Example Value
				5: Failed **6**: Expired **0**: Pending **3:** In Progress **10**: New Request **13**: Processing **20**: Bank Success **21**: Login OK **22**: Login Error 24: Pin Wait **25**: Pin Received **26**: Pin OK **27**: Pin Error **28**: Transferring **29**: Checking
statusDesc		Char (20)	The description of the request status	When status value is 1 - **Completed** 2 - **Rejected** 4 - **Refunded** 5 - **Failed** 6 - **Expired** 0 - **Pending** 3 - **In Progress** 10 - **New Request** 13 - **Processing** 20 - **Bank Success** 21 - **Login OK** 22 - **Login Error** 24 - **Pin Wait** 25 - **Pin Received** 26 - **Pin OK** 27 - **Pin Error** 28 - **Transferring**

Field ID	Field Name	Data Type	Description	Example Value
				29 - **Checking**
requestDate		DATETIME	The fund in / deposit request date Date format: YYYY-MM-DD HH:MM:SS	2020-05-16 18:01:08
transactionType		Char (1)	The transaction type	Possible Value: **D**: Deposit
transactionTypeDesc		Char (20)	The description of the transaction type	When transaction type is D - **Deposit**
currency		Char (3)	Currency Code (ISO 4217). As per the currency from the request	MYR
amount		Decimal (20, 2)	As per the amount from the request	1000.00
transactionDate		DATETIME	The date of status changed Date format: YYYY-MM-DD HH:MM:SS	2020-05-16 18:01:08
remark		Char (255)	The remark input by us	
When status = "28" (Transferring)				
fromBankAcctName		Char (45)	Depositor Name	
transactionAmount		Decimal (20, 2)	The actual amount of the fund transferred	
serviceFee		Decimal (20, 2)		
When status = "1" (Completed)				
designatedBank			When transactionType is D Selected bank to deposit	
bankName		Char (45)	When transactionType is D Selected bank name	
bankBranch		Char (45)	When transactionType is D Selected bank branch	
bankAcctName		Char (45)	When transactionType is D	

Field ID	Field Name	Data Type	Description	Example Value
			Selected bank account holder name	
bankAcctNum		Char (45)	When transactionType is D Selected bank account number	
frombankName		Char (45)	When transactionType is W Bank name that does the fund out	
bankTransactionId		Char (45)	When transactionType is W Reference for the fund out	
transactionFee		Decimal (20, 2)	When transactionType is W Fund out the transaction fee	
transactionAmount		Decimal (20, 2)	The actual amount of the fund transferred	
serviceFee		Decimal (20, 2)		
adjustment			When it is completed but there is input error of service fee/transaction fee. Refer to the Adjustment section below	
When status = "4" (Refunded)				
refundedReason		Char (255)	The reason of the reverse of the fund out.	

Adjustment (in JSON Format)

Field ID	Field Name	Data Type	Description	Example Value
Mandatory				
adjustedDate		DATETIME	The date that we make the adjustment Date format: YYYY-MM-DD HH:MM:SS	
adjustedReason		Char (255)	The reason of the adjustment	
Conditional				

Field ID	Field Name	Data Type	Description	Example Value
serviceFeeBeforeAdjusted		Decimal (20, 2)	This will return the previous value of the service fee when the adjustment is on service fee. The new value is replaced in the **serviceFee** above.	
transactionFeeBeforeAdjusted		Decimal (20, 2)	When transactionType is W This will return the previous value of the service fee when the adjustment is on service fee. The new value is replaced in the **serviceFee** above.	

Example

Response Data when the request is success


```
{
	"status": "success",
	"result": "
sTVAteR6VLBoEwpTq9f2enHJ82frUMA0ZHBJGH23bCw2Sk4qOUhVv5fGVG0OlW4jNMn2zfnYYUmajyTxZBPSjdlnarYBVPROysR1d1zpr2QtClVyUKKtXaR
qdIHMhycrbFHA90GZf\/iZ+rTZAwEcrPx+rwHQyWSTckofGQ8jASaM53LwIIC2HorOE5B33SO+AgbHUZeRKDtUderQYPItIReY5Dshp5092n8Y5ygitvfpX
2UYHdHob0omfVrTaqTIP7LMLfq1FFlAvLtF3H4Tr9Pr+wn+rYZIYlzYjVindCnaZjEFXorhOmU+K6MJrvALcjc4XKFOddgOUuW1gcZlt6rvK+XUE\/gET+D
15phZvutHaW6O9aJ09P\/SaiEZlsw0xPYkWukpkWu2IHQlngks6SuN7tAnK1eE7lwqH0JJPuqbKDMmsLaZ+xLa+AVw10BN3wTJv75I7th\/kfFhO2YvdQ==
"
}
```


Result before encrypted


```
{
	"referenceId":"855112b2-a88b-49a5-8b82-a6b09d7db487",
	"transactionId":"TEST202102031845355805",
	"status":10,
	"statusDesc":"New Request",
	"requestDate":"2021-02-03 18:46:56",
	"transactionType":"D",
	"transactionTypeDesc":"Member Deposit",
	"transactionDesc":"",
	"currency":"MYR",
```

```
      "amount":"100.00",
      "transactionDate":"2021-02-03 18:46:55",
      "remark":""
}
```


Response Data when request is failed


```
{
      "status": "fail",
      "errors": " No Transactions found"
}
```

Appendix A - Message Encryption

Encryption Method : AES128
Block Cipher Mode : CBC

Merchant will be provided the followings when merchant account is ready

1) Merchant Key (16 characters)
2) Merchant Hash IV (16 characters)

1) Encrypt the message string (JSON Format) with AES128, cipher mode CBC
2) Use Base64 to encode the AES128 encrypted text

Example

PHP 7

```
$content = '{
      "merchantId": "94b2522ba609909d",
      "transactionId": "20201030023201612",
      "transactionType": "D",
      "payMethod": "offline",
      "currency": "RMB",
      "amount": "500.00",
      "callback": "https://enmu1r0iut2ws.x.pipedream.net",
      "response": "https://enmu1r0iut2ws.x.pipedream.net",
}';

$secret_key = $merchantKey
$secret_iv = $merchantHashIV;
$mode = "AES-128-CBC";
$encrypted = openssl_encrypt($content, $mode, $secret_key, OPENSSL_RAW_DATA, $secret_iv);
$message = base64_encode($encrypted);
```

JAVA 8

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import javax.crypto.IllegalBlockSizeException;
import javax.crypto.BadPaddingException;
import javax.crypto.NoSuchPaddingException;
import java.security.NoSuchAlgorithmException;
import java.security.InvalidKeyException;
import java.security.InvalidAlgorithmParameterException;
import java.io.UnsupportedEncodingException;
import java.util.Base64;

String content = "{" +
  "\"merchantId\": \"94b2522ba609909d\"," +
  "\"transactionId\": \"20201030023201612\"," +
  "\"transactionType\": \"D\"," +
  "\"payMethod\": \"offline\"," +
  "\"currency\": \"RMB\"," +
  "\"amount\": \"500.00\"," +
  "\"callback\": \"https://enmu1r0iut2ws.x.pipedream.net\"," +
  "\"response\": \"https://enmu1r0iut2ws.x.pipedream.net\"" +
"}";

String secretKey = "abcdefghhgfedcba";
String secretIv = "1234567887654321"; //Merchant Pass1 + Pass2

try {
  Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
  SecretKeySpec keySpec = new SecretKeySpec(secretKey.getBytes("utf-8"), "AES");
  IvParameterSpec ivSpec = new IvParameterSpec(secretIv.getBytes("utf-8"));
  cipher.init(Cipher.ENCRYPT_MODE, keySpec, ivSpec);
  byte[] ciphertext = cipher.doFinal(content.getBytes("UTF-8"));
  String message = Base64.getUrlEncoder().encodeToString(ciphertext);

  System.out.println("message=" + message);
} catch (NoSuchAlgorithmException e) {
  System.err.println(e.getMessage());
} catch (NoSuchPaddingException e) {
  System.err.println(e.getMessage());
} catch (InvalidAlgorithmParameterException e) {
  System.err.println(e.getMessage());
```

```
} catch (BadPaddingException e) {
  System.err.println(e.getMessage());
} catch (UnsupportedEncodingException e) {
  System.err.println(e.getMessage());
} catch (InvalidKeyException e) {
  System.err.println(e.getMessage());
} catch (IllegalBlockSizeException e) {
  System.err.println(e.getMessage());
}
```

Appendix B - Message Decryption

Decryption Method : AES128
Block Cipher Mode : CBC

Merchant will be provided the followings when merchant account is ready

1) Merchant Key (16 characters)
2) Merchant Hash IV (16 characters)

1) Use Base64 to decode the message to return raw message in bytes
2) Decrypt the raw message in bytes with AES128, cipher mode CBC to obtain the JSON plain data.

Example

PHP 7

```
$request = '{
      "merchantId":"82164414b30aa575",
      "message":"Eis\/kISbtVtkLckK\/LHMQQ1kZ9yy0oZV3tk0CPDJv5xfT+xPvbDT510cPL4b8QedGBsWkHqRmpDfbIGyWZVSB3egVdEEXW6I8q6v
xhccIoHruCclCzrIvP9e\/Y1ick1gkMAEZRCTs5jLLvD3waDU+nzMGIyY2oGtn0V0Lu+bSVAjigRDBrwxnullPU2ABvzh9jq4ARAwxjJQPBVIAAeaWTNsqR
i1dvMo4xp6Oq1i9rUafg4DREkizh\/h4p+1a+YwvluuiMnZC4RcCHsiW5CyjPd+Tg6bSkEYFm4Ddh8bNgLgxxlM9U7BTwX2mvSRnLoZotTgflVAva20LiGc
PeM3AllcPuma5khdlAmxaTnFc3uaoKtWPAPEElgldcEZvh\/2Mlra+2t2MfegpcDpquFwr0+Fm+9ZUg2zQUjAx7ry3N6DuFMlKMTJqxs3\/d\/EXvqI\/gL
8K9ntXMnC229140u71UvZyv+jfDr7mgsSwurW5rsIDpZadwKezEsv7Y7KBe9xGPuh+jNev+SqJKytvoG6xwttnqy+K6IPiLFLwXs54GlhKizDVnIamp2cJp
KeGeH+V7C84jzX5f+dXf+yTDkAffUjpD4b0heJM\/7PMwdZqbFho7KHKZyR9OyHkXcHa2J2jN8FB\/egBh0PWncQutxswGUk1+maQWdZkZM5KzF0SRc50kC
vGZPi2vgvKZMcybnXfOttO2Z+PTmd5b0nOtuo5w=="
}';

$content = json_encode($request);
$content = $content->message;

$secret_key = $merchantKey
$secret_iv = $merchantHashIV;
$mode = "AES-128-CBC";
$encrypted = base64_decode($content);
$message = openssl_decrypt($encrypted, $mode, $secret_key, OPENSSL_RAW_DATA, $secret_iv);
```

JAVA 8

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import javax.crypto.IllegalBlockSizeException;
import javax.crypto.BadPaddingException;
import javax.crypto.NoSuchPaddingException;
import java.security.NoSuchAlgorithmException;
import java.security.InvalidKeyException;
import java.security.InvalidAlgorithmParameterException;
import java.io.UnsupportedEncodingException;
import java.util.Base64;

String requestMessage =
  "dO_ih-LJC7fpBjKRTr0Hbmw0rPvn4MKP3XwDPWxPYFY7T09dEze-tar7NXdYqJ0kY-osQs822lyks-
SI3s9NCidEtbulRDtDeBF2Tq0lhMAGseEDgJnyHu6CtSTLCM3iecKYX9qfsK64GndX1wrIAL2mMPeiWNfYN14rowLliqQU8TkvVR2Rd6Ml2XpecL5YXAUsG
Tq8omxFb3YBB5aYoU9n1rh-NzL_43IQJQw-Tx4Kc-eLniTwNJgVUbpYox3vFjrbVeF3pNFZ832nWnXB7RZ0FLCdz77CUXJ4sQdS17z7JgwXmCPxD22-
YybQixlAbP-tSPIebs1i44qiSo7b9asG7d-y20A7ePk3qgLj_PQ=";

String content = "{" +
  "\"merchantId\": \"94b2522ba609909d\"," +
  "\"message\": \"" + requestMessage + "\"" +
"}";

String secretKey = "abcdefghhgfedcba";
String secretIv = "1234567887654321"; //Merchant Pass1 + Pass2

try {
  Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
  SecretKeySpec keySpec = new SecretKeySpec(secretKey.getBytes("utf-8"), "AES");
  IvParameterSpec ivSpec = new IvParameterSpec(secretIv.getBytes("utf-8"));
  cipher.init(Cipher.DECRYPT_MODE, keySpec, ivSpec);
  byte[] requestMessageBytes = Base64.getUrlDecoder().decode(requestMessage);
  byte[] plainText = cipher.doFinal(requestMessageBytes);
  String message = new String(plainText );

  System.out.println("message=" + message);
} catch (NoSuchAlgorithmException e) {
  System.err.println(e.getMessage());
} catch (NoSuchPaddingException e) {
```

```
  System.err.println(e.getMessage());
} catch (InvalidAlgorithmParameterException e) {
  System.err.println(e.getMessage());
} catch (BadPaddingException e) {
  System.err.println(e.getMessage());
} catch (UnsupportedEncodingException e) {
  System.err.println(e.getMessage());
} catch (InvalidKeyException e) {
  System.err.println(e.getMessage());
} catch (IllegalBlockSizeException e) {
  System.err.println(e.getMessage());
}
```